

Database Lab Session 3: Privacy and Anonymization

Sergio Peignier and Nicolas Parisot

Objective of the Lecture The course aims at introducing practical data analysis using two synthetic datasets: one representing pseudonymized hospital records and another representing a leaked commercial dataset containing identity information. The practical session is structured around two conceptual stages. First, by executing a complete re-identification attack on these sets, we will learn firsthand how separate data systems can be combined to reconstruct real-world identities. Then, we will pivot to defense, exploring robust mitigation strategies to safeguard data privacy moving forward.

Lecture Goal

In this lecture, we will adopt the mindset of an ethical attacker to discover how linkage attacks work in practice, ultimately learning how to defend against them using data generalization techniques.

1 The Attacker Mindset

1.1 SQL injection attack

A major risk of database-driven applications comes from the how user input is handled. When an application concatenates untrusted user input directly into a SQL statement, a poorly programmed web form can transform a simple database query into an administrative command.

In a simulated environment, a standard product search bar might execute a query like:

```
SELECT * FROM products WHERE name = 'INPUT';
```

If an attacker inputs the payload `' OR '1'='1'`, the query shifts entirely:

```
SELECT * FROM products WHERE name = '' OR '1'='1';
```

Because `'1'='1'` is always true, the database bypasses the intended search logic and returns every single record within the table, potentially exposing sensitive user credentials, financial records, or system metadata instead of a simple product list.

Exercise 1 Install the `streamlit` library (`pip install streamlit`). Run the `app.py` web application `streamlit run app.py`. This application corresponds to the poorly programmed web form. First run a valid search, typing for example `+33 6 74 27 12 78`. Now based on the previous `SQLi` code, hack the system to get the entire database. Save the database, then you will use to retrieve the medical information from some people from the database in the following part.

1.2 Data Linkage Attack

Data linkage occurs when two or more datasets, which are individually harmless, are combined to reveal sensitive information. Each dataset may appear safe in isolation, but together they can reconstruct identity. This is analogous to assembling two incomplete puzzles that originate from different sources. When correctly aligned, the full picture emerges.

In this course we will work with two datasets. The first contains hospital records including medical diagnoses and genomic variants but no explicit identifiers. The second is a leaked telecommunications dataset that includes real personal identities along with demographic attributes, and that an attacker could have obtained from the dark web.

Both datasets share common fields such as age, gender, and ZIP code, which act as linking quasi-identifiers. When datasets are combined using shared quasi-identifiers, individuals can be matched across datasets. This process effectively transforms de-identified medical records into identifiable information.

The critical insight is that linkage does not require direct identifiers. It relies instead on statistical uniqueness in shared attributes.

The puzzle analogy

Data linkage functions like assembling two incomplete puzzles from different sources; when aligned, the complete, hidden picture emerges.

In this course we simulate a re-identification attack to demonstrate how an individual patient can be identified based on a specific medical condition and genetic mutation, despite the dataset being supposedly anonymized. The simulated re-identification process follows a precise algorithmic execution:

Step 1: Data Alignment: Join the de-identified medical dataset D_m and the leaked telecom dataset D_t over the shared set of Quasi-Identifier attributes QI .

Step 2: Target Filtering: Isolate records based on auxiliary knowledge (e.g., a specific rare genetic mutation or medical diagnosis).

Step 3: Uniqueness Verification: Verify if the intersection yields a singleton ($k = 1$) by grouping the data by the QI attributes. Groups with a size of exactly one represent successfully isolated, unique individuals.

Step 4: Identity Resolution: Map the target's explicit name from the telecom dataset back to their private medical condition.

Algorithmic Re-identification Flow

Privacy leakage is executed via a systematic, four-step technical pipeline: aligning datasets via shared quasi-identifiers (QI), filtering by known rare conditions, verifying statistical uniqueness ($k = 1$), and finally mapping the real name to the sensitive data.

Exercise 2

1. Load `hospital_records.csv` and `telecom.csv` datasets using `pandas` and inspect their content and schema.

2. Identify QIs and homogenize the column names so common columns share the same names.
3. Perform an inner join (merge) between the datasets based on these QIs to construct a linked dataframe, and verify its final shape.
4. Group the linked dataset by the active QIs to calculate the frequency of each unique combination. Identify the number of completely re-identified individuals (where frequency = 1).
5. **Bonus:** Implement a custom filtering function using `.groupby().apply()` to extract the full rows of these unique individuals. Compare how the total number of re-identified individuals changes if 'address' is included in the QIs.
6. Plot a histogram using `matplotlib` to visualize the distribution of the QI group frequencies, setting the number of bins equal to the total number of unique combinations.
7. Simulate a targeted linkage attack by writing a query to isolate the medical profile of a 20-year-old male (`sex = 1`) living in the postal code 69115.
8. Repeat the study with a different set of QIs.

2 The Defender Mindset

Exercise 3 Edit the `app.py` file in order to replace the concatenation-based query by a prepared statement. Run a valid query and try to run again the SQLi and observe the result.

Exercise 4

1. Try modifying manually the dataset to increase the k -anonymity level
2. Use the `anonympy` Python library to perform k -anonymization to protect the `hospital_records` dataset.
3. Try to identify the same individual that was identified in exercise 2.
4. Imagine (or search) strategies to determine the data utility loss after performing k -anonymization.